

O que aconteceu nestes últimos 30 dias?



 Claude Opus 4.6

GPT-5.3-Codex

 Gemini



Engenharia de Prompt



Engenharia de Contexto



Engenharia de Conexão



Engenharia de Conexão

Alavancando a IA em um Mundo Agent-First

Baseado no artigo da OpenAI: Harness Engineering (Fev 2026)

Mauro Widman

Agenda



01 O que é Engenharia de Conexão?



02 O Experimento: Números e Contexto



03 Mudança de Paradigma para Engenheiros



04 Pilar 1 — Projetar o Ambiente



05 Pilar 2 — Especificar a Intenção



06 Pilar 3 — Construir Loops de Feedback



07 Lições Aprendidas e Futuro

O que é Engenharia de Conexão?

Harness Engineering (Engenharia de Conexão) é a disciplina de projetar ambientes, especificar intenções e construir loops de feedback que permitem agentes de IA realizarem trabalho confiável de engenharia de software.

O engenheiro não escreve código

O papel muda: de codificar para orquestrar agentes de IA

O repositório é a interface

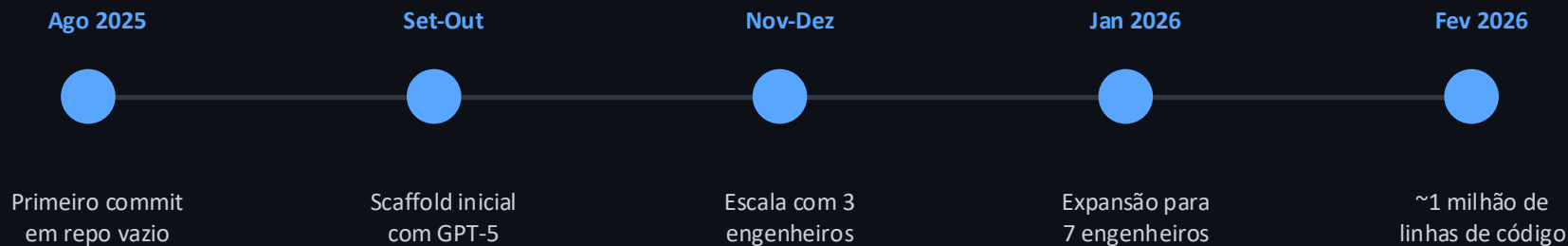
Toda informação que o agente precisa deve estar acessível no repo

Feedback mecânico

Linters, testes e CI validam automaticamente o trabalho do agente

O Experimento da OpenAI

Um produto inteiro construído por agentes Codex — zero código escrito por humanos



O time do Codex Harness na OpenAI queria entender o que muda quando o papel principal de um time de engenharia não é mais escrever código, mas sim projetar ambientes, especificar intenções e construir loops de feedback para agentes.

Os Resultados

~1M

linhas de código

Aplicação, infra, tooling
e documentação

~1.500

pull requests

Abertas e mergeadas
em 5 meses

3,5

PRs/eng/dia

Throughput médio
por engenheiro

0

linhas humanas

Zero código escrito
manualmente

~1/10 do tempo de desenvolvimento comparado ao método tradicional

Mudança de Paradigma

Antes

- ✗ Engenheiro escreve código diretamente
- ✗ Revisões manuais de código
- ✗ Documentação como afterthought
- ✗ Testes escritos por humanos
- ✗ Arquitetura aplicada por convenção

Depois (Agent-First)

- ✓ Engenheiro orquestra agentes
- ✓ Agentes revisam uns aos outros
- ✓ Documentação é a interface principal
- ✓ Agentes geram e rodam testes
- ✓ Arquitetura aplicada por linters/CI

Os Três Pilares da Engenharia de Conexão



Pilar 1

Projetar o Ambiente

Tornar o contexto do projeto legível
para agentes



Pilar 2

Especificar a Intenção

Comunicar o que deve ser feito, não
como fazer



Pilar 3

Construir Loops de Feedback

Validação mecânica e iteração
contínua

Projetar o Ambiente

Do ponto de vista do Codex, informação que ele não consegue acessar em runtime simplesmente não existe.



Repo-Legível

Toda decisão é codificada no repositório como markdown, schemas e planos de execução



UI Legível

Chrome DevTools Protocol integrado ao runtime do agente — DOM snapshots e screenshots



Observabilidade

Logs, métricas e traces expostos via stack de observabilidade local e efêmera



Worktrees Isolados

Cada agente trabalha em uma versão totalmente isolada do app, incluindo logs e métricas

AGENTS.md: Índice, Não Enciclopédia

✗ Abordagem Errada

AGENTS.md monolítico com centenas de linhas — vira uma enciclopédia que:

- Enche o contexto do agente
- Fica desatualizado rapidamente
- Faz o agente pattern-match localmente

✓ Abordagem Correta

AGENTS.md compacto (~100 linhas) que serve como mapa para a docs/ estruturada:

- Aponta para fontes mais profundas
- Navegação intencional
- Documentação viva e modular

Estrutura Recomendada

AGENTS.md (~100 linhas) → docs/architecture/ → docs/design/ → docs/execution-plans/

O próprio AGENTS.md inicial foi gerado pelo Codex — até a documentação é agent-first!

Tornando a UI e Observabilidade Legíveis



Chrome DevTools Protocol

Se o agente está corrigindo um bug de UI, ele precisa ver a UI.

- DOM snapshots em tempo real
- Screenshots automáticos
- Navegação autônoma pelo app
- Comparação pré/pós-tarefa
- Aplicação de fixes em loop



Stack de Observabilidade

Observabilidade como capacidade de runtime, não apenas monitoramento.

- Logs acessíveis diretamente pelo agente
- Métricas locais e efêmeras
- Traces para debug de fluxos
- Stack isolada por worktree
- Tear down automático pós-tarefa

Especificar a Intenção

O papel do humano não desaparece — ele opera em uma camada diferente de abstração.



Priorizar Trabalho

Decidir o que precisa ser feito e em que ordem, traduzindo objetivos de negócio em tarefas para agentes



Traduzir Feedback

Converter feedback de usuários em critérios de aceitação claros e testáveis



Validar Resultados

Revisar e aprovar os outcomes produzidos pelos agentes antes de ir para produção

Princípios Dourados (Golden Principles)

Regras opinativas e mecânicas codificadas diretamente no repositório que mantêm a codebase legível e consistente para agentes futuros.



Pacotes Utilitários Compartilhados

Preferir pacotes compartilhados em vez de helpers escritos à mão — mantém invariantes centralizados



Validação de Fronteiras

Nunca probing de dados 'YOLO-style' — validar limites ou usar SDKs tipados



Processo de Limpeza Recorrente

Manutenção automática e contínua da qualidade do repositório

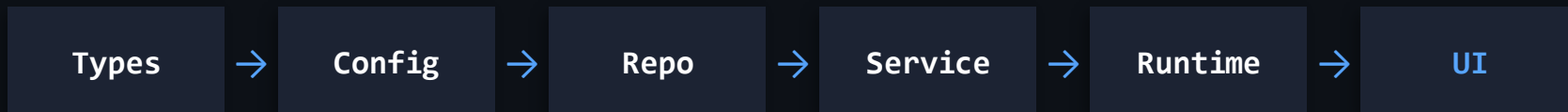


Documentação Cross-Linked

Docs de design e arquitetura interligadas e verificadas mecanicamente

Restrições Arquiteturais e Linters

Fluxo de Dependências (só para frente)



Aplicação Mecânica

Cada domínio de negócio é dividido em camadas fixas. O código só pode depender 'para frente' nesta sequência.

Linters customizados (gerados por agentes!) verificam estas restrições. Se um agente tenta fazer um componente de UI depender diretamente de um repositório de banco de dados, o linter falha o PR e fornece instruções exatas para corrigir a violação arquitetural.

Os próprios linters foram gerados pelo Codex!

Construir Loops de Feedback

A validação mecânica substitui a confiança cega no output do agente.



Testes Automatizados

Suíte de testes gerada e mantida por agentes valida cada mudança



Linters e CI

Verificações automáticas de estilo, arquitetura e qualidade



Ralph Wiggum Loop

Ciclo recursivo: agente executa → revisa → solicita reviews → itera



Feedback Humano

Humanos validam resultados finais e fornecem direcionamento estratégico

O Loop "Ralph Wiggum"

Processo recursivo de feedback onde agentes iteram até que todos os revisores (humanos e agentes) estejam satisfeitos.

1

Executar a tarefa e abrir um Pull Request

2

Revisar suas próprias mudanças localmente (self-review)

3

Solicitar reviews de outras instâncias do Codex

4

Responder ao feedback (humano ou de agentes)

5

Iterar até todos os revisores estarem satisfeitos

↺ Se reviews identificam problemas, volta ao passo 1 — loop contínuo até convergência

Tornando Toda Informação Repo-Legível

O que o agente não consegue ler no repositório, para ele simplesmente não existe.



Decisões de Design

Codificadas como markdown em docs/ com links cruzados entre documentos



Schemas & Tipos

Contratos de interface e modelos de dados definidos no repo, não em wikis externas



Planos de Execução

Passos detalhados para tarefas complexas que guiam o agente passo a passo



Regras Arquiteturais

Constraints e invariantes expressados como testes e linters verificáveis

O Novo Papel do Engenheiro



Arquiteto de Sistemas

Projetar a estrutura macro, definir camadas e fronteiras que os agentes devem respeitar



Curador de Contexto

Manter a documentação, AGENTS.md, schemas e planos de execução atualizados e legíveis



Construtor de Ferramentas

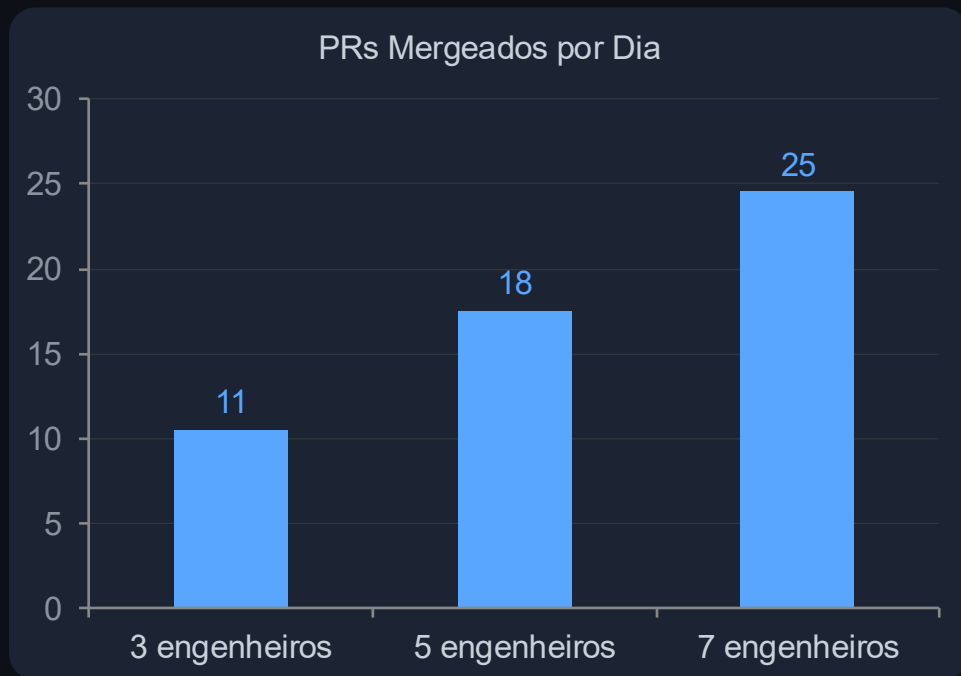
Criar linters, testes e scripts de CI que servem como guardrails para agentes



Revisor Estratégico

Validar outcomes de alto nível, priorizar trabalho e traduzir necessidades de negócio

Throughput e Escala



3,5

PRs por engenheiro por dia
(média ao longo de 5 meses)

~10x

mais rápido que desenvolvimento
tradicional equivalente

Lições Aprendidas



Documentação é produção

Quando agentes são seus principais 'desenvolvedores', a qualidade da documentação determina diretamente a qualidade do software produzido.



Guarda-corpos mecânicos

Linters, testes e CI não são apenas boas práticas — são a única forma de garantir consistência quando agentes geram milhões de linhas de código.



Arquitetura rígida escala

Restrições arquiteturais mecânicas permitem que agentes trabalhem de forma independente sem quebrar invariantes do sistema.



Iteração > Perfeição

Agentes raramente acertam de primeira. O poder está no loop de feedback — iteração rápida e barata converge para qualidade.

Mais Lições e Insights



Agentes escrevem para agentes

Até o AGENTS.md, os linters e a documentação foram gerados pelo próprio Codex — criando um ciclo de auto-melhoria.



O tempo humano é o gargalo

O constraint fixo é o tempo e atenção humana. O investimento vai para capacitar o agente com mais informação e ferramentas.



Runtime legível é essencial

Conectar DevTools e observabilidade ao agente foi uma das decisões mais impactantes do projeto.



Escala linear com engenheiros

Adicionar mais engenheiros aumentou o throughput proporcionalmente, sem overhead de comunicação adicional.

Implicações para o Futuro

O vantagem competitiva não é mais o código — é o controle

Quem dominar a arte de orquestrar agentes terá vantagem sobre quem apenas escreve código

Engenharia de software se torna engenharia de sistemas

O foco passa de 'como implementar' para 'como projetar o ambiente correto'

Documentação e testes ganham protagonismo

Investimento em qualidade de docs e testes tem ROI exponencial com agentes

Times menores, impacto maior

3 engenheiros + agentes produziram o que times muito maiores produziram tradicionalmente



Perguntas?

Obrigado!

Fonte: openai.com/index/harness-engineering/

Mauro